

Reviewer Pack — Minimal Rank of Graphical Motives and Circuit Monotone Width...

Entry 4ae9b39ca1e9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 08:51:41 UTC

Minimal Rank of Graphical Motives and Circuit Monotone Width Inequality

Entry ID: 4ae9b39ca1e9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 08:51:41 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Graphical Motives) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every n -variable Boolean circuit C with monotone complexity at most k , the minimal rank of its associated graphical motive $M(C)$ is upper-bounded by $O(k^2 \log n)$, i.e., $|\text{rank}(M(C)) - O(k^2 \log n)| \leq \varepsilon$ for any $\varepsilon > 0$.

Rationale (proposer's reasoning):

Graphical motives provide a way to encode combinatorial data into algebraic structures, which could potentially reveal underlying complexity-theoretic properties. The conjecture leverages the structure of graphical motives to propose an inequality that relates them to circuit monotone width, a well-studied complexity measure. If true, it would bridge the gap between algebraic and computational complexity theories.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32d16d2a34f06de1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every n -variable Boolean circuit with monotone complexity at most k , if the inequality $|\text{rank}(M(C)) - O(k^2 \log n)| \leq \varepsilon$ holds for all seeds and circuits, where ε is a predefined threshold and $O(k^2 \log n)$ is the expected upper bound on rank.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, k):
        # Simple DPLL-based solver to generate a monotone circuit
        if n == 1 and k == 0:
            return []
        elif n == 1 and k == 1:
            return [[random.choice([1, -1])]]
        else:
            literals = [i for i in range(1, n+1)]
            random.shuffle(literals)
            circuit = []
            for _ in range(k):
                clause = [literals.pop()]
                while len(clause) < 2 and literals:
                    clause.append(literals.pop())
                circuit.append(clause)
            return circuit

    def compute_graphical_motive(circuit):
        # Simplified graphical motive computation
        n = len(circuit[0])
        M = [[0] * n for _ in range(n)]
        for clause in circuit:
            for lit1 in clause:
                for lit2 in clause:
                    if abs(lit1) != abs(lit2):
                        M[abs(lit1)-1][abs(lit2)-1] += 1
        return M

    def matrix_rank(M):
        # Gaussian elimination to compute the rank of a matrix
        n = len(M)
```

```

rank = n
for i in range(n):
    if M[i][i] == 0:
        reduce = False
        for k in range(i+1, n):
            if M[k][i] != 0:
                M[i], M[k] = M[k], M[i]
                reduce = True
                break
        if not reduce:
            rank -= 1
            continue
    for j in range(n):
        if i != j and M[j][i] != 0:
            factor = -M[j][i] / M[i][i]
            for k in range(n):
                M[j][k] += factor * M[i][k]

return rank

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
min_rank = float('inf')
max_n = 0

for n in n_values:
    for _ in range(5):
        k = random.randint(0, n)
        circuit = generate_circuit(n, k)
        M = compute_graphical_motive(circuit)
        rank = matrix_rank(M)
        instances_tested += 1
        min_rank = min(min_rank, rank)
        max_n = max(max_n, n)

epsilon = 0.1
conjecture_holds = abs(min_rank - (k**2 * math.log(n))) <= epsilon

return {
    "metric_name": "min_rank",
    "metric_value": min_rank,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_rank = sum(r["metric_value"] for r in results) / len(results)
std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e2c4283.py", line 104, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e2c4283.py", line 80, in run_trial
    M = compute_graphical_motive(circuit)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e2c4283.py", line 46, in compute_graphical_motive
    M[abs(lit1)-1][abs(lit2)-1] += 1
    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from determining whether the conjecture is supported or falsified. | next: Investigate the cause of the crash and attempt to run the test again. If the crash can be resolved, re-evaluate the conjecture based on the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14187
2	preregistration	ollama_remote	glm4:latest	0	0	9455
3	novelty	ollama_remote	glm4:latest	0	0	8533
4	novelty	ollama_remote	glm4:latest	0	0	8474
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14231
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10080
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10428
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12147
9	judge	ollama_remote	glm4:latest	0	0	15298

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102833 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4ae9b39ca1e9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ae9b39ca1e9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ae9b39ca1e9.tar.gz` (if generated)